

Web VAPT Section 05 - Burp Suite Manual Request Analysis

Professional documentation for HTTP interception, request modification, Repeater testing, and evidence handling.

Enterprise Security Assessment Lab

PROJECT AREA	Web Application Vulnerability Assessment and Penetration Testing
PRIMARY TARGET	DVWA running in local lab environment
TESTING ENVIRONMENT	Kali Linux attack machine with Docker-based vulnerable target
PREPARED BY	Jagriti Banerjee
DOCUMENT TYPE	Individual Web VAPT Attack-Section PDF

AUTHORIZED LAB TESTING ONLY

1. Document Purpose

This individual PDF documents the Web VAPT attack section titled 'Burp Suite Manual Request Analysis'. It is designed for the Enterprise Security Assessment Lab GitHub portfolio and describes the objective, tooling, commands, sanitized output, evidence mapping, security interpretation, remediation, and redaction requirements for this specific section.

This document is written as a public portfolio version. It does not contain live client data, production system details, unredacted credentials, session cookies, raw hash dumps, or destructive attack instructions.

Field	Details
Project Area	Web Application Vulnerability Assessment and Penetration Testing
Primary Target	DVWA running in a local lab environment
Testing Environment	Kali Linux attack machine with Docker-based vulnerable target
Testing Style	Reconnaissance -> Enumeration -> Manual validation -> Exploitation proof -> Evidence capture
Methodology Reference	OWASP Web Security Testing methodology, OWASP Top 10, and PTES-style workflow
Prepared By	Jagriti Banerjee

2. Section Overview

Item	Details
Section Number	05
Section Title	Web VAPT Section 05 - Burp Suite Manual Request Analysis
Risk / Severity Style	Manual Validation Phase
OWASP Mapping	Supports multiple OWASP categories by enabling parameter, cookie, and state-changing request analysis.
Primary Evidence Count	2

2.1 Objectives

- Intercept HTTP requests and inspect parameters.
- Review cookies, session values, and security level indicators.
- Modify requests safely in Repeater.
- Create evidence for manual validation of injection and request manipulation.

2.2 Tools Used

Tool	Purpose / Use in this section
Burp Suite Proxy	Request capture and HTTP history review.
Burp Repeater	Controlled request modification.
Browser	Manual verification and proof screenshots.
DVWA	Intentionally vulnerable application used for validation.

3. Methodology and Execution Workflow

The execution workflow follows a controlled lab approach: confirm authorization and target scope, run only the tools required for the section, save outputs, validate observations manually where applicable, capture screenshots, redact sensitive values, and document findings without exaggeration.

3.1 Command Reference

Browser proxy: 127.0.0.1:8080
Burp Suite -> Proxy -> HTTP history
Burp Suite -> Send to Repeater

Example request pattern:
GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Host: 127.0.0.1:8081
Cookie: PHPSESSID=<REDACTED>; security=low

3.2 Expected / Sanitized Output

Burp captured target HTTP requests.
Requests were sent to Repeater and modified safely.
Session cookies were identified as sensitive and marked for redaction.

3.3 Validation Criteria

- The target must be the local DVWA or authorized lab environment only.
- The output must match the intended section objective.
- Evidence screenshots must clearly show the validation result or tool output.
- Any sensitive values must be blurred, cropped, removed, or replaced with placeholders.
- The section must not claim findings that are not supported by evidence.

4. Evidence Mapping

Evidence File	Evidence Type / Reason
13-burp-intercepted-request.png	Burp intercepted request evidence.
14-burp-repeater-modified-request-response.png	Burp Repeater request modification evidence.

5. Professional Security Analysis

Burp Suite is the central tool for professional web application testing because it makes request/response behaviour visible.

Repeater allows controlled manual testing without relying only on automated scanners.

The intercepted request evidence supports later SQLi, XSS, CSRF, and parameter tampering documentation.

5.1 Why This Section Matters

This section matters because professional VAPT is not only about running a tool. It requires a clear objective, controlled execution, evidence capture, correct interpretation, and practical remediation. For recruiters and reviewers, this PDF shows that the work was structured, documented, and aligned with real security methodology.

5.2 Common Mistakes to Avoid

- Uploading raw outputs that expose cookies, hashes, paths, or credentials.
- Claiming production-level impact without production context.
- Reporting scanner output as a confirmed vulnerability without validation.
- Using unclear screenshots that do not show the relevant result.
- Mixing unrelated phases in one evidence file without explanation.

6. Risk Analysis

- Cookies or session identifiers exposed in screenshots can leak authenticated access.
- State-changing requests may reveal missing CSRF protection.
- Parameters may accept unsafe input if not validated server-side.

6.1 Real-World Impact Statement

The real-world impact depends on the affected asset, authentication context, data sensitivity, privilege level, exposed functionality, and compensating controls. Because this project uses DVWA in a lab, severity is described as a real-world context estimate rather than a formal client CVSS score.

7. Remediation and Hardening Guidance

- Use HttpOnly, Secure, and SameSite cookie attributes.
- Validate all parameters server-side.
- Use CSRF protection on state-changing actions.
- Never publish unredacted session data.

7.1 Verification After Remediation

- 1 Re-run the original test case in the lab or development environment.
- 2 Confirm that the previous vulnerable behaviour no longer appears.
- 3 Check server logs for blocked or rejected attempts.
- 4 Capture new evidence showing the fix or defensive control.
- 5 Update the report status from validated/open to fixed/retested if applicable.

8. GitHub Redaction and Publishing Checklist

Cookies, hashes, credentials, session IDs, shell paths, database output, and private details must be redacted before public upload.

Item	Action Before Upload
PHPSESSID / Cookies	Blur or replace with <REDACTED_COOKIE>.
Password hashes	Blur or replace with <REDACTED_HASH>.
Cracked passwords	Do not publish; replace with <REDACTED_PASSWORD>.

Item	Action Before Upload
Database dumps	Crop, blur, or summarize instead of uploading raw data.
Webshell paths/files	Do not upload shell source files; redact paths if needed.
Local usernames/paths	Blur if private or unnecessary.
Tool raw outputs	Upload only sanitized outputs or summary documents.

8.1 Safe Git Commands Before Push

```
git status
git diff --cached
git diff --cached | grep -iE "password|secret|token|cookie|PHPSESSID|hash|credential"
```

9. Conclusion

This document completes the individual Web VAPT PDF for Section 05. It documents the section objective, execution workflow, evidence mapping, professional interpretation, risk, remediation, and GitHub safety checks. The content is aligned to the actual Enterprise Security Assessment Lab Web VAPT evidence set and avoids unsupported production claims.

Legal and ethical notice: This documentation is for authorized lab testing only. Do not apply these techniques to public or third-party systems without explicit written permission.